Ámbito

Introducción



- El ámbito se refiere a la sección de código donde algo nombrado mediante un identificador (usualmente objetos o variables) vive (puede ser recuperado)
- Cuando un ámbito desaparece todo lo que se definió dentro de ese ámbito también desaparece
- El ámbito es diferente a los modificadores de acceso
- El ámbito es diferente al tiempo de vida de un objeto
- El ámbito es algo general que aplica a cualquier identificador (clase, método, propiedad, variable), no sólo a elementos de objetos (como los modificadores) y establece no sólo acceso sino también tiempo de vida pero solo de identificadores, no establece el tiempo de vida de objetos.

Introducción



- Existen los siguientes niveles de ámbito léxico:
  - Global: acceso desde cualquier punto del programa
  - Objeto: acceso desde dentro de una clase
  - Método: acceso desde dentro de un método
  - Bloque: acceso desde dentro de un bloque de control

Ámbito global



- En Java también se le conoce como ámbito estático
- Los elementos creados en un ámbito global desaparecen hasta que el programa termina
- Las clases (a menos que se definan dentro de otra clase) tienen este ámbito
- Los elementos marcados con la palabra reservada **static** tienen un ámbito global

Ámbito global



- Cuando se tiene un elemento estático no es necesario contar con una instancia de clase para tener acceso al elemento (no se necesita de un objeto), aunque también se puede acceder al elemento desde una instancia. La buena práctica es siempre acceder a través del nombre de la clase
- Una forma de crear variables globales (compartidas en todo el programa) es definiendo dentro de alguna clase una propiedad publica estática
- CUIDADO: el uso de variables globales es una mala práctica de programación

Ámbito de objeto



- Las propiedades y métodos se definen sintácticamente dentro de una clase y viven dentro del ámbito del objeto una vez es creado
- Cuando el objeto desaparece su ámbito muere y con él todo lo que se definió en su ámbito
- Sintácticamente este ámbito comienza cuando se abren llaves después de la definición de clase y termina cuando se cierran
- Para hacer referencia explícita a este ámbito puede utilizase la palabra reservada **this**

Ámbito de método



- Sólo se pueden definir variables dentro de este ámbito
- Una vez el método termina, todos los identificadores que se definieron dentro de él también desaparecen (no así los objetos que pudieran estar referenciados por otro identificador con mayor ámbito)
- Los parámetros que recibe un método tienen ámbito de método
- Sintácticamente este ámbito comienza cuando se abren llaves después de la definición del método y termina cuando se cierran

Ámbito de bloque



- Surge cuando se utiliza alguna sentencia de control (if, while, for, etc.)
- Dentro de este ámbito sólo se pueden definir variables
- Una vez el bloque termina todos los identificadores que se definieron dentro de ´el desaparecen (igual que en el ámbito de método)
- Los ámbitos de bloque suelen estar demarcados por llaves, aunque existen excepciones sintácticas

Más detalles sobre ámbito



- Los ámbitos se pueden anidar. Ejemplo: dentro de un bloques se puede definir otro bloque, dentro de una clase se pueden definir métodos, etc.
- Dos identificadores no se pueden llamar igual si están en el mismo ámbito (si están en diferente ámbito si)
- Al hacer referencia a un identificador, siempre se trata de recuperar desde el ámbito más anidado primero, sino se encuentra se va al siguiente nivel de anidación

Mejores prácticas en el uso de ámbitos



- Siempre declara un identificador en el ámbito más cercano a donde será utilizado
- NUNCA uses variables globales, a menos que las declares como constantes, con esto evitas efectos colaterales
- Ten cuidado con el ámbito a nivel de clase/objeto cuando manejes atributos, los métodos que modifican el estado de un atributo deben declarar de forma directa esta intensión
- Un buen uso de ámbito reduce el acoplamiento y en consecuencia la complejidad

Código amigable a la recolección de basura



- La recolección de basura se refiere a la liberación automática de memoria que ya no es utilizada por el programa
- Este mecanismo automatizado lo tienen distintas plataformas de ejecución como son la máquina virtual de Java o la máquina virtual de Python
- Los lenguajes donde el programador debe manejar manualmente la memoria (como C o C++) no cuentan con este mecanismo
- El recolector de basura es un proceso especial, independiente al programa, que analiza la memoria, cuando detecta que un bloque de memoria no es referenciado por alguna referencia (variable o atributo en POO) libera esa memoria

Código amigable a la recolección de basura



- El ámbito y la recolección de basura están estrechamente relacionados
- Siempre que un ámbito termina, todas las referencias creadas en ´el mueren
- En consecuencia, mucha memoria puede liberarse
- Entre mayor sea el tiempo de vida de el ámbito de una referencia, menos probable es que se libere la memoria
- En ocasiones la memoria ya no se necesita pero quedó referenciada en un ámbito que sigue vivo
- La buena práctica es utilizar referencias en el ámbito con el tiempo de vida más corto posible
- Evitar usar el ámbito global también por esta razón